

Detailed Code Analysis: `script.py`

This document provides a comprehensive, plain-English walkthrough of the entire `script.py` script. The script functions as a hybrid security tool combining **Machine Learning (SVM)**, **Generative AI (Groq/Llama 3)**, **Email Protocol Clients (IMAP & SMTP)**, and a **Flask Web Dashboard** to implement an interactive scambaiting system.

Table of Contents

1. General Architecture
 2. Imports and Environment Setup
 3. Global Settings and Configurations
 4. AI Model Loading and Core AI Logic
 5. Email Extraction and SMTP Client Utilities
 6. Email Processing Pipeline (`process_inbox`)
 7. Flask GUI Web Server & Background Workers
 8. Main Execution Wrapper
-

1. General Architecture

The script handles the entire lifecycle of email interception and automated scambaiting. The pipeline operates as follows:

1. **Fetch:** Connect to your Gmail mailbox via IMAP and search for unread emails from the last 48 hours.
 2. **Feature Extraction:** Parse the email body and extract 40 semantic/structural features.
 3. **Classify:** Normalize features using a pre-fitted scaler (`StandardScaler`) and predict spam status and probability using a local Radial Basis Function SVM (`SVC`).
 4. **Draft:** If classified as spam (Nigerian Prince, phishing, etc.), query the Groq LLM API using Llama 3.3 to write an auto-reply. Save it as a draft in history.
 5. **Approve / Send:** Wait for the user to click "Approve" on the Web GUI, then dispatch the response via SMTP and mark the conversation status as "sent".
-

2. Imports and Environment Setup

At the very top of `script.py`, the script sets up its dependencies:

- **Standard Protocol Modules:** `imaplib` (for fetching email from server folders) and `smtplib` (for sending email replies).
- **Email Parser Modules:** `email` classes to decode mime envelopes and extract plain text or HTML bodies.
- **Python Utilities:** `json` for saving scambait history, `re` for regular expression checks, `os` for resolving file paths, and `threading` for background email monitoring.
- **Machine Learning Dependencies:**
 - `pickle`: To load the pre-trained `.pkl` model files.

- **numpy & pandas:** To build and format numerical feature vectors for inputting to the classifier.
 - **Dynamic Imports:** The script dynamically appends the **Q2** directory to Python's system path (**sys.path**) to import the preprocessing code (**extract_email_features** and **FEATURE_COLUMNS**) from **Q2/preprocess_question2.py**.
-

3. Global Settings and Configurations

This section configures connection credentials and directories:

- **SMTP/IMAP Server Hosts:** Defined as Gmail endpoints (**smtp.gmail.com** on port 587 and **imap.gmail.com**).
 - **EMAIL_ACCOUNT / EMAIL_PASSWORD:** Credentials used to authenticate email connections.
 - **GROQ_API_KEY:** Token used to authenticate Groq API requests.
 - **SPAM_FOLDER_NAME:** Name of the spam folder (adjusted to handle Greek characters in Gmail).
 - **HISTORY_FILE:** Path to **scambait_history.json** where all scam communications are saved.
-

4. AI Model Loading and Core AI Logic

A. Model Loading

At startup, the script searches for **spam_model.pkl** and **scaler.pkl** in the script directory.

- If found, they are loaded into global variables **CLASSIFIER_MODEL** and **FEATURE_SCALER**.
- If missing, the script prints a warning and falls back to running classification via the Groq API (backward compatibility).

B. **classify_email(email_text)**

This is the core classifier function:

1. **Extract Body:** It parses the email text to isolate the actual body from the subject lines.
2. **Run Preprocessing:** It calls **extract_email_features(body)** to compute the 40 handcrafted metrics.
3. **Format DataFrame:** It packs the dict values into a pandas **DataFrame** aligned with the exact column names the scaler expects, avoiding Scikit-Learn warnings.
4. **Predict & Probabilities:** It transforms features using **FEATURE_SCALER.transform(X)** and predicts the label using **CLASSIFIER_MODEL.predict(X_scaled)**. If the model supports probability estimation, it queries **predict_proba** to calculate the exact scam probability.
5. **LLM Fallback:** If local files fail to load, the script sends the email body to Groq's **llama-3.1-8b-instant** with a JSON-formatted system prompt prompting it to decide spam or ham.

C. **generate_scam_reply(...)**

This queries Groq's **llama-3.3-70b-versatile** to produce a scambaiting response.

- It uses a custom persona ("Alex Johnson") designed to sound eager and interested, asking for a lot of details (to waste the scammer's time) while refusing to provide any real personal information.
- It supports **conversational history** for multi-turn chats by packing the previous messages list directly into the messages array sent to the API.

5. Email Extraction and SMTP Client Utilities

- `extract_text_from_email(msg)`: Iterates through multipart email sections to extract plain text. If only HTML is present, it uses regular expressions and HTML unescaping to clean style blocks, script tags, and HTML tags, returning plain text.
 - `send_reply(to_address, original_subject, reply_body)`: Log in to Gmail's SMTP server, activates TLS encryption (`starttls`), logs in, packages the text as an e-mail reply (matching the subject line prefixing with `Re:`), and sends it to the scammer.
-

6. Email Processing Pipeline (`process_inbox`)

This is the main email processor loop:

1. **Select Folders**: Scans all available email folders (dynamically filtering out Sent, Trash, or Draft folders).
 2. **Search Unread**: Queries the IMAP server for unread messages received within the last 48 hours.
 3. **Parse & Deduplicate**: Loops over the found email IDs, fetches headers, identifies the sender, and formats a unique thread identifier: `scammer_address | normalized_subject`.
 4. **Active Thread Bypass**: If this thread key already exists in `scambait_history.json`, it bypasses the classifier and treats it as an ongoing scambait (Spam).
 5. **Classify**: If not active, runs `classify_email`.
 6. **Handle Spam (Nigerian Prince scenarios)**:
 - Logs stats.
 - Appends the email to history, noting the exact `scam_probability`.
 - Generates the reply draft via Groq.
 - Saves the reply to history with status `"pending_approval"`.
 - Marks the email as seen and copies it to Gmail's Spam folder so that it is cleared from the inbox and won't get checked again.
 7. **Handle Ham**: Bypasses any action for legitimate emails.
-

7. Flask GUI Web Server & Background Workers

The Flask app acts as the user control panel, running on port 5000:

- `home()` (`/`): Serves `templates/gui.html` (the Web Telegram-like UI).
 - `api_history()` (`/api/history`): Returns `scambait_history.json` content as JSON.
 - `check_status()` (`/api/check_status`): Returns whether the background checking thread is active.
 - `check_mail()` (`/api/check_mail`): Starts the `process_inbox()` pipeline inside an asynchronous background thread (`threading.Thread`) using a mutual exclusion lock (`checking_lock`) to prevent concurrent mailbox checks.
 - `approve_reply()` (`/api/approve_reply`): Receives the conversation key, extracts the draft reply from history, dispatches it via SMTP, and marks the status as `"sent"`.
 - `reject_reply()` (`/api/reject_reply`): Receives the key. It can delete the pending reply draft, or if `delete_thread` is specified (e.g. false positive), it deletes the conversation thread entirely from history.
-

8. Main Execution Wrapper

The `if __name__ == "__main__":` entry point controls how the program starts:

- **CLI Mode:** Running `python script.py --cli` checks the inbox once in the terminal and exits.
- **GUI Mode (Default):** Running `python script.py` starts the Flask server, launches a 1.2-second background timer to automatically open the default web browser to the GUI, and listens for requests.